

Asynchronous Reconciliation based on Operational Transformation for P2P Collaborative Environments

Michelle Cart, Jean Ferrié
LIRMM, University Montpellier 2, CNRS
161, rue Ada - 34392 Montpellier (France)
{cart, ferrie}@lirmm.fr

Abstract—Reconciling divergent copies is a problem encountered in distributed systems, groupware, version control systems and personal work involving several mobile computing devices. Published reconciliation methods, whether synchronous or asynchronous, require some ordering facility provided either by a central component (master copy, sequencer) or by a fully distributed mechanism (timestamps, state vectors, ...). Nevertheless, scalability is limited. This paper presents an asynchronous algorithm based on Operational Transformations which provides the means to reconcile any number of copies, without this limitation. Copies can be modified (concurrently or not) and then reconciled pair-wise, at any time, regardless of the pair, while their convergence is achieved. Its main advantage is thus to enable free propagation of update operations while ensuring they will be ordered in the same global order.

Keywords; replication; copy consistency; reconciliation; history merge; operational transformation

I. INTRODUCTION

Users involved in mobile computing, concurrent engineering or distributed collaborative work, generally work on copies of shared objects. For instance, in mobile computing a user can replicate an object (calendar, file, address book, etc.) on mobile devices (Laptop, PDA, etc.) before disconnection and then manipulate these copies in disconnected work and on different devices. In asynchronous or multi-synchronous [7] collaborative groupware and distributed engineering, each user manages its private copy of the shared object and propagates its updates to the others (or makes them public) when connected. In both cases, as each copy of the object can be modified separately and independently of the others, copies could diverge and therefore have to be reconciled [25].

Reconciling multiple distributed copies of an object can be obtained in a synchronous or asynchronous mode. In synchronous methods, copies are reconciled in real time. Among these, methods based on Operational Transformations [8], and developed for collaborative (CSCW) and real-time environments [24, 30, 26, 27, 28, 34, 29, 14, 15, 10, 16, 17]

maintain copy consistency of an object by memorizing operations in histories and exploiting their semantic properties. Every operation generated by a user is immediately executed on his copy before being broadcast to all the other copies in order to be executed on them. Causal reception is generally needed to respect causal ordering relation between operations. Moreover as concurrent operations are not necessarily executed in the same order on each copy, they need to be ordered (by means of timestamps, state vectors or sequencer) and then transformed before being executed using Operational Transformation, to achieve copy convergence. In these methods, all the copies play the same role and no copy is favoured. Unfortunately, they are not suitable for P2P environments because of the ordering mechanisms they use.

With asynchronous methods, reconciling multiple copies involves pair-wise copy reconciliation. The published methods generally require a *master copy*, also called primary or public copy. Every copy therefore must be separately reconciled with the master copy to achieve copy convergence. Examples of such methods are given by Configuration Management Environments [2, 6, 9], most of which are based on the *Copy/Modify/Merge* paradigm. Version control systems based on this principle were specified to reconcile text files [2, 31], XML files [33] or graphical objects [11, 12]. Copies are reconciled using state of their representation [2, 31, 33, 11] and more recently history and semantic properties of operations [12].

A synchronizer is another example of asynchronous method. It enables *two copies* of the same object to be reconciled in order to obtain an integrated view of the object. It takes two divergent copies of the object as an input and then returns the copies in the same state, after somehow merging the updates. File Synchronizers (Microsoft's Briefcase, Power Merge, Windows File Synchronizer, Unisson [3], etc.) consider the object to be a file hierarchy. They allow create/delete actions on files and directories, and also updates, except when they concern two copies of the same file, to be

propagated from one copy of the file system to another. Solving the conflict is delegated to the user when updates concern two copies of the same file. Data Synchronizers (Palm Pilot Hotsync, Puma Technology Intellisync, Microsoft ActivSync, Apple I-Sync, etc.) allow reconciling and merging of updates relative to two copies of the same file (calendar, address book, etc.), while delegating to the user conflicting situations which generally correspond to non-commutative actions. Several merge tools [19] presently used in the field of collaborative work and also in software engineering can be considered as Data Synchronizers – they allow two copies of the same file to be reconciled in accordance with the data type (text [2, 31], UML or XML [33]), after having detected divergences using, for instance, Diff algorithms [4, 19]. A generic data synchronizer was proposed in [20]. This synchronizer is built on the adaptation of an algorithm which was initially proposed in the context of distributed real-time collaborative (synchronous) environments [34] and uses Operational Transformations [8].

All of these merging and synchronizing tools, which were initially designed for reconciling two copies, can be used for reconciling n copies (with $n > 2$). However they assume either that a master copy exists and that each copy is reconciled with the master copy, or that a total order between operations is built dynamically by means of a sequencer [20], otherwise copy convergence will not be achieved.

In Bayou [23], the anti-entropy protocol permits any pair of copies to be reconciled without using a master copy. This protocol uses a version vector (i.e. state vector) associated with each copy to detect concurrent operations. Besides, a *particular site* chosen when the system starts is required to dynamically construct a total order between committed operations. Moreover, the reconciliation needs to undo and redo out of order received operations.

A reconciliation platform called IceCube [13], was proposed in order to reconcile divergent copies and to merge their histories using semantic properties of operations executed on these copies. The IceCube algorithm is centralised at a *primary site*. It computes (in a combinatorial manner) optimal merges, minimizing the number of rejected operations and constructs a total order between committed operations. This requires all concurrent operations to be sent to this site for merging and resulting merged history to be dispatched to all sites. Consequently, all sites must be connected during reconciliation.

The fact that either a master copy or a central component (particular site, sequencer) is required to (pair-wise) reconcile copies of the same object is incompatible with a P2P environment in which no copy should be favoured. Indeed, a P2P environment offers a completely decentralised approach for sharing objects by permitting replication of copies of the same object from site to site. Unfortunately, in existing P2P

systems, there is only partial or no copy consistency, i.e. when a copy is updated the modification is not propagated to all the other copies. Although this aspect is not important for musical files, it may be for other applications. In popular P2P systems involving millions of users, i.e. Gnutella and Kazaa, manipulated objects are read-only files, so updates are not taken into account. In Freenet [5], updates are partially taken into account insofar as they are propagated from origin sites to neighbour sites when connected. In P-Grid [1], update propagation is ensured by an algorithm offering probabilistic guarantees for copy consistency. However, the main drawback is that updates on copies of an object can only come from the origin site of this object, i.e. from a master copy. Nevertheless, the WOOT [22] and the APPA [18] systems have been recently proposed in the context of P2P collaborative environments, in order to concurrently update copies of an object while ensuring their convergence. Reconciliation in the APPA system lies on the P2P network layer that enables providing a simulated *common storage* to the replication service. This storage is used by peers to store local actions (with associated constraints) after their execution on replicas; it is also used by reconcilers to retrieve these actions and then produce a global schedule which will be executed at each peer. This system needs some complex synchronization between participating peers. As for the WOOT system, it is specially adapted to a linear structure. It does not use either a state vector mechanism or a primary site but the ordering relation induced by the operations on each element (i.e. character) of the structure is locally maintained at each copy and associated to broadcast operations.

In this paper, we present two asynchronous merge algorithms using Operational Transformations. The first algorithm, called MOT1, assumes the existence of a master copy. It is presented as a preliminary step to understand the use of Operational Transformations as well as the requirement concerning the operations ordering. The second algorithm, called MOT2, achieves reconciliation without a master copy. It allows copy consistency, *without discriminating in favour of any particular copy and without requiring either state vectors or a central component*. Even though the copies may have been modified (concurrently or not) on different sites, the algorithm we present, can reconcile them pair-wise, regardless of the pair, and copy consistency is always ensured. The main advantage of this algorithm is that it allows *free and lazy propagation* of updates originating from the different copies while ensuring they will be ordered in the *same global order*. Thus, it is particularly suitable for P2P environments in which no copy is favoured.

The paper is set up as follows. Section 2 defines causal precedence and concurrence relations between operations and recalls the principle of Operational Transformations. Section 3 presents the merge algorithm, called MOT1, that ensures copy reconciliation while requiring a master copy. Section 4

highlights the limits of MOT1 when no master copy is available. Then it outlines the principles and the algorithm, called MOT2, which enables any two copies to be reconciled and therefore any number of copies, without favouring any particular copy, but while ensuring their convergence. Some properties of histories produced by MOT2 are described and MOT2 proof is provided. Finally, related work is compared in section 5.

II. UNDERLYING MODEL

In the following, we consider an object (i.e. text, graphics, file, file system, XML tree, calendar, etc.) can be handled using definite operations $\{op\}$ and is replicated on different sites. Initially, the copies are identical and correspond to the same object state, then they progress independently of each other on their own site, which causes them to diverge. With each copy C_i of the object, the history H_i of operations that have been executed on C_i is associated.

A. Relationships between Operations

The history H_i , associated with the copy C_i at site S_i , memorizes the sequence of operations that transforms the copy C_i from its initial state into its current state, when executed in this order on copy C_i . More precisely, an history H_i is constituted by elements such as $\langle S_{op}, ns, op \rangle$ where op is an operation executed on the copy associated with H_i , S_{op} the site where the operation was generated and ns a sequence number local to S_{op} . Operations stored in the history H_i at site S_i have been either generated at S_i and therefore immediately executed on S_i , or were generated at another site S_j and executed on S_i as a result of reconciling S_i with S_j or with another site.

Two operations either are related by a causal ordering relation or are concurrent.

Definition 1. Given any two operations op_k and op_l , generated at sites S_{op_k} and S_{op_l} we say that op_k *causally precedes* op_l (noted $op_k \rightarrow_C op_l$) iff: (i) $S_{op_k} = S_{op_l}$ and op_k was generated *before* op_l , or (ii) $S_{op_k} \neq S_{op_l}$ and op_l was generated *after* the execution of op_k by site S_{op_l} , or (iii) there is an operation op_m such that $op_k \rightarrow_C op_m$ and $op_m \rightarrow_C op_l$.

Definition 2. Operations op_k and op_l are said to be *concurrent* (noted $op_k // op_l$) iff: not ($op_k \rightarrow_C op_l$) and not ($op_l \rightarrow_C op_k$).

The order of operations in H_i expresses *precedence*.

Definition 3. Given two operations op_k and op_l in the history H_i , op_k is said to *precede* op_l (noted $op_k \rightarrow_{H_i} op_l$) iff op_k appears *before* op_l in H_i .

Let us note that *precedence* is compatible with the causal ordering relation, i.e. if $op_k \rightarrow_C op_l$ then $op_k \rightarrow_{H_i} op_l$. However, two operations may belong to the same history as a

result of a reconciliation, without having a causal precedence relation between them.

B. Operational Transformation

Operational Transformation is used to achieve copy convergence when concurrent and non-commutative¹ operations are executed on copies in different orders. Two transformation functions, called forward transposition and backward transposition [26, 28] were introduced. In the following, only forward transposition is used. To illustrate this concept, let us consider two copies of an object, C_1 and C_2 , that are in the same initial state. Let us suppose that operation op_1 [resp. op_2] executed on copy C_1 [resp. C_2] leads to the state $C_1.op_1$ [resp. $C_2.op_2$]. Later, the execution of op_2 [resp. op_1] on copy C_1 [resp. C_2] leads to the state $C_1.op_1.op_2$ [resp. $C_2.op_2.op_1$]. When operations are not commutative, the resulting states are not equal ($C_1.op_1.op_2 \neq C_2.op_2.op_1$). In these conditions, the forward transposition allows transformation of an operation before its execution so that it takes all concurrent operations serialized before it into account. In short, the forward transposition function, denoted in the following as $Transpose_forward(op_1, op_2)$, is specific to a couple of concurrent operations (op_1, op_2) which are defined from the same object state. As a result, it gives the operation op_2' (also written as $op_2^{op_1}$) which has the same effect as op_2 but which is defined from the state resulting from the execution of op_1 , and op_2' is called the forward transposition of op_2 with op_1 . To guarantee copy convergence, forward transpositions must meet condition TP1 [8, 24] which is summed up by state equality: $\forall C_i, C_i.op_1.op_2' = C_i.op_2.op_1$. Moreover, they generally must meet a further condition TP2 [24] which is not detailed, and be conformed to the TTF model [21]. This model adds a layer above the object representation that keeps track of elements removed from the object state. By this way, forward transposition can easily be defined while satisfying both conditions TP1 and TP2 and respecting the user's intention.

Example 1. Let us assume that object C is represented as a string of characters and the operation $insert(p, x)$ inserts character x at position p in the string. Copies C_1 and C_2 are in the same initial state "effect". The operation $op_1 = insert(2, 'f')$ executed on copy C_1 adds 'f' at position 2, whereas operation $op_2 = insert(6, 's')$, which is concurrently executed on copy C_2 , adds 's' at the end of the string. Execution of these operations in different orders leads to copies in divergent states: $C_1.op_1.op_2 = "effectf"$ and $C_2.op_2.op_1 = "effects"$. To guarantee copy convergence, operation op_2 [resp. op_1] has to be forward transposed with op_1 [resp. op_2] before being executed on copy C_1 [resp. C_2] so as to take the concurrent operation op_1 [resp. op_2] serialized before it into account, in order to

¹ That concerns the "forward" commutativity [35], as opposed to the "backward" commutativity.

obtain: $C_1.op_1.op_2' = C_2.op_2.op_1' = \text{"effects"}$. The transposition function to be used and meeting condition TP1 is as follows:

```

Transpose-forward(insert( $p_1, x_1$ ), insert( $p_2, x_2$ )) =
  case  $p_1 ? p_2$  of
     $p_1 < p_2$  : return insert( $p_2+1, x_2$ ) ;
     $p_1 > p_2$  : return insert( $p_2, x_2$ ) ;
     $p_1 = p_2$  : if  $x_1 = x_2$  then return id ;
                  elsif range( $x_2$ ) > range( $x_1$ )
                    then return insert( $p_2, x_2$ ) ;
                  else return insert( $p_2+1, x_2$ ) ; endif ;
  endcase.

```

The forward transposition of $op_2 = \text{insert}(6, 's')$ with op_1 here would be $op_2' = \text{Transpose-forward}(\text{insert}(2, 'f'), \text{insert}(6, 's')) = \text{insert}(7, 's')$, whereas the forward transposition of $op_1 = \text{insert}(2, 'f')$ with op_2 would give the operation $op_1' = op_1$ as a result.

III. RECONCILIATION USING A MASTER COPY

In the reconciliation model using a master copy, the master site, that we designate by S_M , maintains the master copy as well as the history H_M of the operations applied to it from the initial state to obtain the current state. By construction, the master copy is the most up-to-date copy among all copies of the object. Any site S_i ($S_i \neq S_M$) that wants to reconcile its copy must merge its history H_i with the history H_M of the master copy. More precisely S_i has to obtain from S_M the operations missing in H_i and to *integrate* them into H_i . In the same way S_M has to obtain from S_i the operations missing in H_M and to *integrate* them into H_M .

A. Principle of Integration

The integration procedure which is the basis of our proposals was initially proposed in SOCT4 [34]. It assumes that operations are memorized in the histories associated with the copies according to a global unique order. It is executed by a site S_i whenever an operation op generated at a remote site has to be integrated into H_i . It determines the operation, noted op^{seq} , to be executed on the current state of the local copy and inserts the received operation op into the history H_i at the range corresponding to its position in the global order (see Fig.1). The integration of operation op in position t is based on the assumption that op is defined from the state resulting from the execution of operations op_1 to op_{t-1} and that all operations located from position t (sequence seq) are concurrent to op .

The *Integration* procedure defined below is called to integrate the remote operation op , received as a triplet $\langle S_{op}, ns, op \rangle$, in position t in history H_i of site S_i . It delivers as a result the operation op^{seq} to be executed on the current state of the copy C_i .

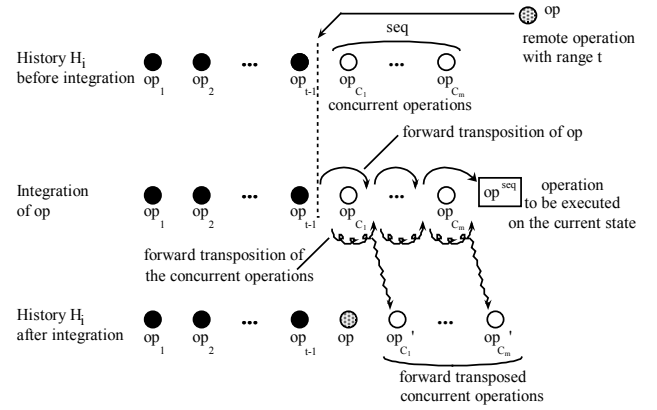


Figure 1. Integration of an operation

Notations. H_i : representation of the history by a table of items in the form $\langle S_{op}, ns, op \rangle$;
 $H_i(k)$.operation corresponds to the field op of $H_i(k)$
 $SizeofH_i$: number of items in H_i

procedure Integration ($H_i, t, \langle S_{op}, ns, op \rangle, op^{seq}$);

```

-- 1. shift the history to insert  $\langle S_{op}, ns, op \rangle$  in position  $t$ 
  for  $k := sizeofH_i$  downto  $t$  do  $H_i(k+1) := H_i(k)$ ; end for;
   $H_i(t) := \langle S_{op}, ns, op \rangle$ ;  $sizeofH_i := sizeofH_i + 1$ ;
-- 2. determine the operation to be executed on the current
  state and transform operations that follow  $op$  in  $H_i$ 
  for  $k := t+1$  upto  $sizeofH_i$  do
     $op_k := H_i(k).operation$ ;
     $H_i(k).operation := \text{Transpose-forward}(op, op_k)$ ;
     $op := \text{Transpose-forward}(op_k, op)$ ;
  end for;
   $op^{seq} := op$ ;
end Integration;
```

Along step 2, the operation op is forward transposed with each operation of seq ; the resulting operation, noted op^{seq} , will be executed on the current state of the copy. During the calculation of op^{seq} , each operation of seq is transposed to take the insertion of op in H_i into account. More precisely, considering $seq = op_{c_1}.op_{c_2} \dots op_{c_m}$, each operation op_{c_i} of seq is forward transposed with $op^{seq_{i-1}}$, where $seq_{i-1} = op_{c_1}.op_{c_2} \dots op_{c_{i-1}}$ is the sequence of operations in seq that precede op_{c_i} . So, operation op_{c_i} is replaced in H_i by $op_{c_i}.op^{seq_{i-1}}$ with $op^{seq_0} = op$. In the *Integration* procedure, $\text{Transpose-forward}(op, op_k)$ is the function that delivers $op_{c_i}.op^{seq_{i-1}}$ and $\text{Transpose-forward}(op_k, op)$ is the function that delivers op^{seq_i} , assuming that $i=k-t$.

The memorized history H_i does not correspond to the real history (i.e. the sequence of operations actually executed on the local copy) but it is equivalent to it. The fact that operations are memorized in all histories according to a global unique order entails that the histories memorized on the different sites converge towards the same history and are finally identical when all operations have been integrated within all sites. Note that the integration of an operation in

the last position in H_i does not require any transposition. It only involves memorizing a new item. For clarity, we introduce the *Append* procedure, which in the following enables us to distinguish the integration with transposition from the integration without transposition.

Finally the validity conditions of the integration procedure may be expressed as follows. Given a history $H_i = H_C.seq$, the integration of operation op into H_i , after H_C and before seq , and the execution of op^{seq} are correct if conditions (a) to (c) have been met:

- (a) : operations are memorized in all histories according to a global unique order;
- (b) : op is defined from the state produced by the execution of operations in H_C ;
- (c) : $\forall op' \in seq$, op' is concurrent to op (i.e. $\text{not } (op \rightarrow_c op')$ and $\text{not } (op' \rightarrow_c op)$).

The history resulting from the integration of op is $H_C.op.seq'$, where seq' means that seq operations have been forward transposed with op to take the insertion and execution of op into account.

B. The MOT1 Algorithm

The principle of the reconciliation using a master copy consists in merging any history H_i with the history H_M of the master site. Before the merge, the histories H_M and H_i are identical until index k_S , where k_S corresponds to the last operation of the history obtained from the last reconciliation of sites S_M and S_i . So we have: $H_i(j) = H_M(j)$, $\forall j : 1 \leq j \leq k_S$. Let us call H_C this common sub-history (common prefix) of H_M and H_i and let us designate by h_M [resp. h_i] the part of H_M [resp. H_i] that follows H_C . So we get: $H_M = H_C.h_M$ and $H_i = H_C.h_i$. More precisely, h_i corresponds to the sequence of operations generated at S_i since the last reconciliation of S_i with S_M and h_M corresponds to the operations generated at S_M or at another site that was reconciled with S_M . The h_M operations are concurrent to the h_i operations. Merging histories H_M and H_i involves two phases. Each one is a straightforward application of the Integration procedure. Phase (1) proceeds on site S_i and makes the pair (H_i, C_i) progress. It involves importing h_M and integrating it into H_i from position $k_S + 1$. The obtained history is: $H_i = H_C.h_M.h_i'$ where h_i' is the sub-history h_i modified by forward-transposition to take h_M operations into account. Moreover each operation returned by the integration procedure is executed on the current state of C_i . Phase (2) proceeds on site S_M and makes the pair (H_M, C_M) progress. It involves importing h_i' and appending (integrating without transposition) it to H_M so as to finally obtain $H_M.h_i'$. Each operation in h_i' is executed on the current state of C_M .

The principle described above is fulfilled by the history merging algorithm MOT1 ("Merge based on Operational Transformation"). MOT1 is composed of two parts. MOT1_A

corresponds to the phase (1) processing and is performed on any site S_i ($S_i \neq S_M$). It accepts as inputs the pair (C_i, H_i) , (abbreviated to the site name S_i) and the sub-history h_M composed of the operations missing in H_i . As output, it produces an up-to-date pair (C_i, H_i) .

```

procedure MOT1A ( $S_i, h_M$ );
-- Phase (1): progression of  $(C_i, H_i)$ 
-- Determine the index  $k_S$  reached after the previous
-- merging of sites  $S_M$  and  $S_i$ 
 $k := k_S + 1$ ;
for  $t$  in  $1.. \text{sizeof } h_M$  loop           -- each operation of  $h_M$ 
   $\langle S_{op}, ns, op \rangle := h_M(t)$ ;           -- is integrated into  $H_i$ 
  Integration ( $H_i, k, \langle S_{op}, ns, op \rangle, op^{seq}$ );
  execute ( $op^{seq}, C_i$ ); -- then executed on the current state
   $k := k + 1$ ;
endloop;
end MOT1A;

```

MOT1_B corresponds to the phase (2) processing and is performed on site S_M . As an input, it accepts the site name S_M (i.e. the pair (C_M, H_M)) and the sub-history h_i' of the missing operations in H_M . As output, it produces an up-to-date pair (C_M, H_M) .

```

procedure MOT1B ( $S_M, h_i'$ );
-- Phase (2): progression of  $(C_M, H_M)$ 
for  $t$  in  $1.. \text{sizeof } h_i'$  loop
   $\langle S_{op}, ns, op \rangle := h_i'(t)$ ;           -- each operation of  $h_i'$ 
  Append ( $H_M, \langle S_{op}, ns, op \rangle$ ); -- is appended to  $H_M$ 
  execute ( $op, C_M$ ); -- and executed on the current state
endloop;
end MOT1B;

```

With MOT1, the roles of sites S_M and S_i are not symmetrical. They are fixed *once and for all* in-so-far as the master site stays the same for all reconciliations. It can also be noticed that sites that get reconciled by MOT1 have to transmit to each other the missing operations only. Example 2 illustrates a succession of reconciliations of sites S_2 and S_3 with the master site S_1 , by using MOT1 algorithm. The following notations are used. $\text{Synch}(S_i, S_j)$ means reconciliation of sites S_i and S_j , where S_i is the master site; $\text{Synch}(S_i, S_j) \Rightarrow$ introduces the history resulting from reconciliation of S_i and S_j . The sequence of operations generated at site S_i between its $(n-1)^{\text{th}}$ and n^{th} reconciliation is designated by $h_i[n]$; $h_i[n]'$ means that operations in $h_i[n]$ have been forward transposed during the reconciliation of S_i with another site (merging $h_i[n]$ with another history).

Example 2. Initially, all copies of the object are in the same state; then sites progress independently from each other (see Fig. 2). The history resulting from the first reconciliation between S_2 and S_1 is: $\text{Synch}(S_1, S_2) \Rightarrow h_1[1].h_2[1]'$ (italics are used to highlight sequences of operations that have been modified by transposition during the reconciliation). Then S_2 and S_1 continue to progress independently. Further reconcilia-

tion of S_3 and S_1 leads to the resulting history: $\text{Synch}(S_1, S_3) \Rightarrow h_1[1].h_2[1]'.h_1[2].h_3[1]'$, where $h_3[1]$ has been forward transposed with the master site history. Just before reconciling S_2 and the master site S_1 , the respective histories of S_2 and S_1 are: $H_2=h_1[1].h_2[1]'.h_2[2]$ and

$$H_1 = h_1[1].h_2[1]'.h_1[2].h_3[1]'.h_1[3].$$

Reconciling S_1 and S_2 leads to the result: $\text{Synch}(S_1, S_2) \Rightarrow h_1[1].h_2[1]'.h_1[2].h_3[1]'.h_1[3].h_2[2]'$. All copies converge towards the *same* state and will be identical when, in the absence of newly generated operations, all sites get reconciled with the master site S_1 (after S_3 has been reconciled with S_1 , in the absence of new operations generated by S_2 , S_3 and S_1).

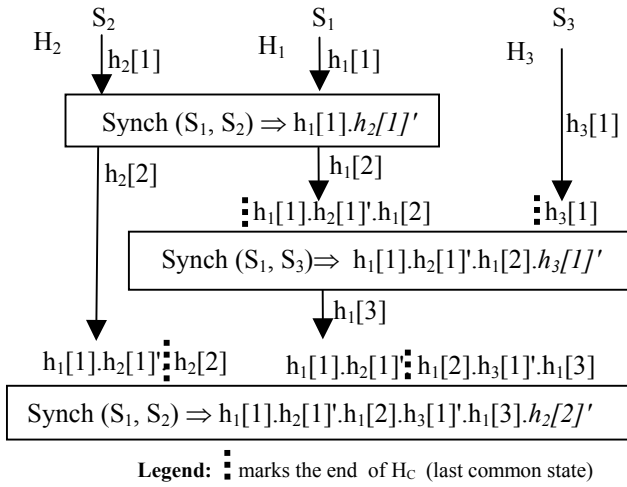


Figure 2. An example of reconciliation by MOT1 with the master site S_1

IV. RECONCILIATION WITHOUT A MASTER COPY

The principle of reconciliation without a master copy assumes that sites get synchronized and merge their histories, in pairs, *regardless of the pair associations*, while finally ensuring *copy convergence*.

A. Limitations of MOT1

We will provide an example to illustrate the effect of using MOT1 to achieve reconciliation without a master copy and demonstrate the impossibility of obtaining copy convergence. In order to apply MOT1, it is necessary to define the role of each S_i and S_j site, before each reconciliation $\text{Synch}(S_i, S_j)$, since this role is not symmetrical. The role assigned to a site is temporary and only set for the duration of the reconciliation. As a convention, the notation $\text{Synch}(S_i, S_j)$ means that when reconciling sites S_i and S_j , the first S_i site serves as the master site (S_M).

Example 3. Let us consider the four sites S_1, S_2, S_3 and S_4 represented in Fig. 3. The histories resulting from successive synchronizations with MOT1 are:

$$\begin{aligned} \text{Synch}(S_1, S_3) &\Rightarrow h_1[1]; \text{Synch}(S_2, S_4) \Rightarrow h_2[1]; \\ \text{Synch}(S_2, S_3) &\Rightarrow \mathbf{h_2[1]}.h_2[2].\mathbf{h_1[1]'.h_3[2]'}; \end{aligned}$$

$$\text{Synch}(S_1, S_4) \Rightarrow \mathbf{h_1[1].h_2[1]'.h_4[2]}'.$$

It can be noted that histories H_1 (or H_4) and H_2 (or H_3) have no common prefix. Their last common state is the initial state. However, some operations (shown in bold) are present in both histories although they appear under *different forms* because they have been transposed. Moreover, these operations appear in *different orders* in each history. In these conditions it is no longer possible to reconcile again S_1 and S_3 (by either $\text{Synch}(S_3, S_1)$ or $\text{Synch}(S_1, S_3)$) when using MOT1. Applying this merge algorithm would result in a history in which some operations would appear twice – which of course is incoherent.

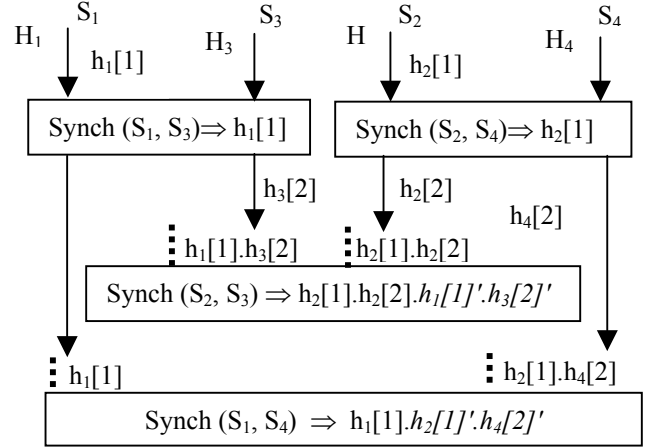


Figure 3. Reconciling attempt using MOT1 without a master copy

In the following, sequences in the same order of operations that are identical in their identity (fields $\langle S_{op}, ns \rangle$ are identical) but different in their form (field $\langle op \rangle$ is different) are called *avatars*. Avatars are characterized by the fact that they are not defined from the same state in both histories. In example 3, $h_2[1]$ in H_2 and $h_2[1]'$ in H_1 are avatars as well as $h_1[1]$ in H_1 and $h_1[1]'$ in H_2 . A predetermined total order between sites could be used to assign their role to sites that achieve reconciliation. During a reconciliation $\text{Synch}(S_i, S_j)$, the S_i site serving as the master site would be such that $S_i < S_j$. The order $S_1 < S_2 < S_3 < S_4$ was implicitly used in example 3. Even when totally ordering the reconciling sites, MOT1 does not ensure that common operations would appear in the same order in the histories.

B. General Principle of MOT2

In the absence of a master copy, the MOT2 merge algorithm presented in the following ensures copy convergence while permitting any pair of copies to be reconciled. To obtain this property, MOT2 assumes an ordering relation is defined among sites. This relation can be obtained from the site names (i.e. IP addresses) provided that names are unique and not re-allocated. It can also be obtained by identifying each copy with a unique name at its creation; this name may be deduced from the name of the copy provided for the

creation, by using a hierarchical identification schema. This ordering relation is used by MOT2 to *serialize operations according to their generator sites*. A unique global order between the operations can thus be dynamically built without requiring a centralizing or ordering mechanism (timestamp, state vector, sequencer, etc.). As a result, histories produced by MOT2 are such that the common sub-histories appear in the *same order*. Moreover, in MOT2, the role of sites is totally *symmetrical* when reconciling. In particular, the history produced by Synch(S_i, S_j) is identical to that produced by Synch(S_j, S_i).

The basic principle of MOT2 is as follows. Given the input histories: $H_i = H_C.op_i$ and $H_j = H_C.op_j$ where op_i and op_j are operations generated by sites S_{op_i} and S_{op_j} , respectively, the resulting history when reconciling S_i and S_j is:

if $S_{op_i} < S_{op_j}$ then Synch(S_i, S_j) $\Rightarrow H_C.op_i.op_j'$
 if $S_{op_i} > S_{op_j}$ then Synch(S_i, S_j) $\Rightarrow H_C.op_j.op_i'$
 where op_i' [resp. op_j'] means that operation op_i [resp. op_j] has been forward transposed with operation op_j [resp. op_i].

More generally, given the input histories:
 $H_i = H_C.op_i.seq_i$ and $H_j = H_C.op_j.seq_j$ where op_i and op_j are operations respectively generated by sites S_{op_i} and S_{op_j} , seq_i and seq_j are sequences of operations generated by any sites, the choice of the operation (op_i or op_j) to integrate after H_C depends on the generator sites and involves the following effects:

if $S_{op_i} < S_{op_j}$ then integration of op_i in H_j , which gives:
 $H_i = H_C.op_i.seq_i$ (H_i unchanged) and
 $H_j = H_C.op_i.op_j'.seq_j'$
 if $S_{op_i} > S_{op_j}$ then integration of op_j in H_i , which gives:
 $H_i = H_C.op_j.op_i'.seq_i'$ and
 $H_j = H_C.op_j.seq_j$ (H_j unchanged)

where $op_i'.seq_i'$ [resp. $op_j'.seq_j'$] means that operations $op_i.seq_i$ [resp. $op_j.seq_j$] have been forward transposed with the operation op_j [resp. op_i].

C. The MOT2 Merge Algorithm

The principle described above is achieved by the MOT2 history merging algorithm. MOT2 may be executed by any site S_i . It accepts any history H_j as input, and reconciles its copy C_i with C_j by merging histories H_i and H_j . For the moment we assume that the whole history H_j is available to S_i . The possibility of transmitting to S_i only the part of H_j that follows the prefix common to H_i and H_j is studied in 4.5.

procedure MOT2 (S_i, H_j);

-- Look for the prefix H_C common to H_i and H_j : determine
 -- the index k_s of the last operation of H_C
 $k := k_s + 1$;

while ($k \leq \text{sizeof}(H_i)$) **and** ($k \leq \text{sizeof}(H_j)$) **loop**

$< S_{op_i}, ns_i, op_i > := H_i(k)$;

$< S_{op_j}, ns_j, op_j > := H_j(k)$;

case $S_{op_i} ? S_{op_j}$ **of**

$S_{op_i} < S_{op_j}$: -- op_i has to be integrated into H_j

Integration ($H_j, k, < S_{op_i}, ns_i, op_i >, op_i^{seq}$);

$S_{op_j} < S_{op_i}$: -- op_j has to be integrated into H_i

Integration ($H_i, k, < S_{op_j}, ns_j, op_j >, op_j^{seq}$);
 execute (op_j^{seq}, C_i); -- and executed

$S_{op_j} = S_{op_i}$: -- operation is already present in H_i and H_j

endcase;

$k := k + 1$;

endloop; -- the end of H_i or H_j has been reached

while $k \leq \text{sizeof}(H_j)$ **loop** -- end of H_i : append the

$< S_{op_j}, ns_j, op_j > := H_j(k)$; -- remainder of H_j to H_i

Append ($H_i, < S_{op_j}, ns_j, op_j >$);

execute (op_j, C_i);

$k := k + 1$;

endloop;

end MOT2;

MOT2 begins by determining the prefix common to H_i and H_j . Then, the generator sites of operations that follow the common prefix in H_i and H_j are compared in order to determine the operation to be integrated. When the compared operations op_i and op_j are such that $S_{op_j} < S_{op_i}$, that means operation op_j is missing in H_i ; so it has to be integrated into H_i and executed on the current state of the copy C_i after having been forward transposed with operations following it in H_i . When the compared operations op_i and op_j are such that $S_{op_i} < S_{op_j}$, that means operation op_i is missing in H_j . It has to be integrated into H_j in order that operations following it in H_j were transposed. After integration, the common prefix is augmented by one operation and the process is repeated until the end of one of the histories. The remaining operations of H_j are then appended to H_i . When the compared operations op_i and op_j are such that $S_{op_i} = S_{op_j}$, they have the same generator site and are therefore identical (i.e. $ns_i = ns_j$ and $op_i = op_j$), which means that the operation is common to both histories H_i and H_j . Therefore, one directly skips to integrate the next operation. The following example illustrates processing of the algorithm and presents a situation where histories produced by MOT2 have common non-contiguous operation sequences.

Example 4. Let us consider the four sites S_1, S_2, S_3 and S_4 represented in Fig. 4. The site order is: $S_4 < S_3 < S_2 < S_1$. The histories produced by the MOT2 algorithm after successive reconciliations are given below.

Synch (S_2, S_1) $\Rightarrow h_1[1]$.

Synch (S_3, S_1) $\Rightarrow h_3[1].h_1[1]'.h_1[2]'$.

Synch (S_2, S_3) $\Rightarrow h_3[1].h_1[1]'.h_2[2]'.h_1[2]'''.h_3[2]'$.

Synch (S_4, S_1) $\Rightarrow h_4[1].h_3[1]'.h_1[1]'''.h_1[2]'''.h_1[3]'$.

Before once more reconciling sites S_2 and S_1 , their histories are as follows: $H_1 = h_4[1].h_3[1]'.h_1[1]'''.h_1[2]'''.h_1[3]'$ and

$H_2 = h_3[1].h_1[1]'.h_2[2]'.h_1[2]'''.h_3[2]'.h_2[3]$.

Note that H_2 and H_1 contain several avatars (shown in bold), and these appear in the same order. Reconciling S_2 and S_1 using MOT2 successively achieves the following statements. We underline the resulting common sub-history.

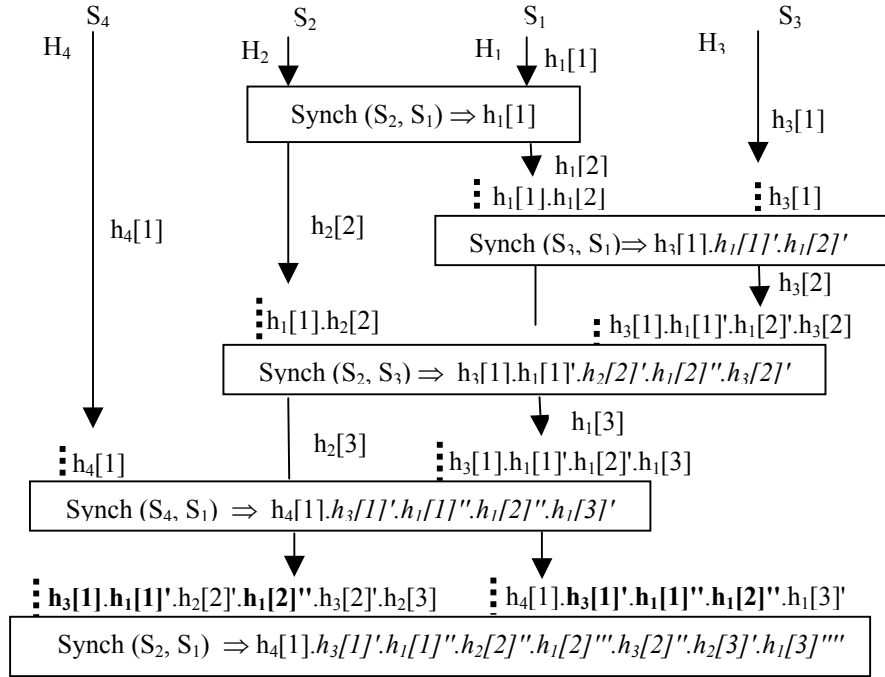


Figure 4. Example of reconciliation using MOT2

a. Integrate $h_4[1]$ into H_2 , which gives:

$H_1 = h_4[1].h_3[1]'.h_1[1]'.h_1[2]'.h_1[3]'$ (unchanged) and

$H_2 = h_4[1].h_3[1]'.h_1[1]'.h_2[2]'.h_1[2]'''.h_3[2]'.h_2[3]'$;

b. Integrate $h_2[2]''$ into H_1 , which gives:

$H_1 = h_4[1].h_3[1]'.h_1[1]'.h_2[2]'.h_1[2]'''.h_1[3]'$ and

$H_2 = h_4[1].h_3[1]'.h_1[1]'.h_2[2]'.h_1[2]'''.h_3[2]'.h_2[3]'$ (unchanged);

c. Integrate $h_3[2]''$ into H_1 , which gives:

$H_1 = h_4[1].h_3[1]'.h_1[1]'.h_2[2]'.h_1[2]'''.h_3[2]'''.h_1[3]'''$ and

$H_2 = h_4[1].h_3[1]'.h_1[1]'.h_2[2]'.h_1[2]'''.h_3[2]'.h_2[3]'$ (unchanged);

d. Integrate $h_2[3]'$ into H_1 , which gives:

$H_1 = h_4[1].h_3[1]'.h_1[1]'.h_2[2]'.h_1[2]'''.h_3[2]'''.h_2[3]'.h_1[3]'''$ and

$H_2 = h_4[1].h_3[1]'.h_1[1]'.h_2[2]'.h_1[2]'''.h_3[2]'''.h_2[3]'$ (unchanged);

e. Append $h_1[3]'''$ into H_2 , which gives:

$H_1 = H_2 = h_4[1].h_3[1]'.h_1[1]'.h_2[2]'.h_1[2]'''.h_3[2]'''.h_2[3]'.h_1[3]'''$;

Finally we get: $\text{Synch}(S_2, S_1) \Rightarrow$

$h_4[1].h_3[1]'.h_1[1]'.h_2[2]'.h_1[2]'''.h_3[2]'''.h_2[3]'.h_1[3]'''$.

In Example 4, we observe that the common sequences of operations, either contiguous or not, appear in the *same order* in all histories. This is actually the main feature of MOT2. While allowing free propagation of histories, since a site may at any time reconcile its copy with any site, MOT2 guarantees that sub-histories common to various histories appear in the *same order* (see Theorem 5). By means of successive synchronizations, a *global order* is built without requiring any centralized component nor ordering mechanism. For this reason, MOT2 is particularly well suited to P2P environments where copies may be concurrently modified without discriminating in favour of any copy or site while converging towards the same state.

MOT2 enables epidemic propagation of operations while respecting *local order* to each site as well as *global order* (between generator sites). According to a general result proved in [32], these properties are sufficient to ensure propagation respecting *causal order* between operations, without using either particular protocol layer (i.e. causal reception) or extra mechanisms (timestamp, state vector, etc.).

D. MOT2 Properties and Proof

This section presents the properties and theorems that enable us to prove the correction of the MOT2 merge algorithm. Demonstrations of the theorems are given in the Appendix. The subsequent results we are interested in concern the operation order. Consequently, although the operations may appear under different forms because of the transpositions, we will combine the operation and its various transposed forms in the notation. In other words, the notation op_k will represent either the generated operation op_k or its forward transposed forms op_k' , op_k'' , etc. The fact that an operation got transformed through transpositions therefore no longer appears in the formulations.

Property P1. Given H_i , $\forall op_k \in H_i$ and $\forall op_l \in H_i$ such that: $op_k \rightarrow_{H_i} op_l$, after merging H_i with another history using MOT2, the precedence in the resulting history H is still: $op_k \rightarrow_H op_l$.

As (see Definition 3) the precedence is compatible with the causal ordering relation, merging preserves both the precedence ($\rightarrow_H$) and the causal precedence relation ($\rightarrow_c$).

Let us now consider the histories H_i and H_j of sites S_i and S_j with a common prefix H_C . Let us call op_i and op_j the operations belonging to histories H_i and H_j , respectively, and defined from the same state (the state left by H_C). Then we have: $H_i = H_C.op_i.seq_i$ and $H_j = H_C.op_j.seq_j$, where seq_i and seq_j are sequences of any operations. The precedence between op_i and op_j in the history resulting from the merge of H_i and H_j is determined by the order between their generator sites. Property P2 expresses this result.

Property P2. Given op_i and op_j such that: $H_i = H_C.op_i.seq_i$ and $H_j = H_C.op_j.seq_j$, after merging H_i and H_j using MOT2, the precedence in the resulting history H is such that:

$$\begin{aligned} \text{if } S_{op_i} < S_{op_j} & \text{ then } op_i \rightarrow_H op_j \\ \text{if } S_{op_i} > S_{op_j} & \text{ then } op_j \rightarrow_H op_i. \end{aligned}$$

The following property generalizes property P2 to the case of a sequence of operations.

Property P3. Given $op_i \in H_i$ and a sequence $seq \in H_j$, defined from the *same* state and such that $H_i = H_C.op_i.seq_i$ and $H_j = H_C.seq.seq_j$, after merging H_i and H_j using MOT2 the precedence in the resulting history H is such that:

$$\text{if } \forall op \in seq: S_{op} < S_{op_i} \text{ then } seq \rightarrow_H op_i.$$

Given two operations op_k and op_l that occur consecutively in a history H produced by MOT2, the following theorem specifies that if they are concurrent then they are ordered according to their generator site.

Theorem 1. Given two operations op_k and op_l that occur consecutively in a history H produced by MOT2 and such that $op_k \rightarrow_H op_l$: if $op_k // op_l$ then $S_{op_k} < S_{op_l}$.

Theorem 2, obtained by contraposition, specifies that if the operations are both consecutive and ordered in the history according to the inverse order of their generator site, then they are related by a causal precedence relation.

Theorem 2. Given two operations op_k and op_l that occur consecutively in a history H produced by MOT2 and such that $op_k \rightarrow_H op_l$: if $S_{op_k} \geq S_{op_l}$ then $op_k \rightarrow_C op_l$.

These theorems are illustrated by Example 4, when each sequence $h_i[n]$ corresponds to a single operation. The final history is:

Synch (S_2, S_1) $\Rightarrow h_4[1].h_3[1].h_1[1].h_2[2].h_1[2].h_3[2].h_2[3].h_1[3]$
We observe (see Fig. 4) that consecutive concurrent operations ($h_4[1]$ and $h_3[1]$, $h_3[1]$ and $h_1[1]$, $h_2[2]$ and $h_1[2]$, $h_2[3]$ and $h_1[3]$) are actually ordered according to their generator site order (recall that in this example the site order is: $S_4 < S_3 < S_2 < S_1$). Concerning consecutive operations that are ordered according to the inverse order of their generator site ($h_1[1]$ and $h_2[2]$ on one hand, $h_1[2]$ and $h_3[2]$ on the other), they are actually related by a causal precedence relation: $h_1[1] \rightarrow_C h_2[2]$ and $h_1[2] \rightarrow_C h_3[2]$.

Theorem 3 is a generalization of Theorem 1 and Theorem 4 is deduced by contraposition from Theorem 3

Theorem 3. Given the sequence $seq.op$ in a history H produced by MOT2, where seq is constituted from operations concurrent to operation op , then: $\forall op_k \in seq \Rightarrow S_{op_k} < S_{op}$.

Theorem 4. Given the sequence $seq.op$ in a history H produced by MOT2 where $seq = op_0...op_n$. If sequence seq contains an operation op_k such that $S_{op_k} \geq S_{op}$ then:

$$\exists op_l \in \{op_k...op_n\} \Rightarrow op_l \rightarrow_C op.$$

Finally, theorem 5 states that, when histories H_i and H_j produced by MOT2 contain common operations, then these appear in the *same* order.

Theorem 5. Given op_k and $op_l \in H_i$, op_k and $op_l \in H_j$, where H_i and H_j are histories produced by MOT2:

$$\text{if } op_k \rightarrow_{H_i} op_l \text{ then } op_k \rightarrow_{H_j} op_l.$$

This theorem underlies the proof of the MOT2 algorithm. Indeed, as operations common to histories produced by MOT2 appear in the *same* order in these histories, we are sure that, whatever successive synchronizations occur, the histories which have integrated the same operations are *identical*. In other words, by means of synchronizations, histories associated with each copy integrate new operations and therefore converge towards the *same* history. So the MOT2 algorithm dynamically guarantees operation ordering according to a unique *global order* without requiring a centralising or ordering (timestamp, state vector, sequencer, etc.) mechanism. Therefore, it naturally scales to a large number of sites in general P2P environments, without needing extra-mechanisms.

E. Limiting the transmitted history

This section describes how the remote history needed during reconciliation can be shortened. The site, let us say S_i , that initiates synchronization transmits to the other site S_j a summary of its history H_i . This summary consists of a list of items $\langle S, n \rangle$ elaborated from H_i . Each sequence $h_k[m]$ in H_i is compacted into an item $\langle S_k, n \rangle$ where S_k is the generator site of $h_k[m]$ and n the number of operations in $h_k[m]$. From this summary site S_j can determine the prefix common to S_i and S_j and transmit back to S_i (a) the summary corresponding to this common prefix followed by (b) the normal history (i.e. a list of items $\langle S_{op}, ns, op \rangle$) from the common prefix. Site S_i , in its turn, will transmit to S_j the operations that follow the common prefix in the history H_i . Then MOT2 can be executed on each site independently.

V. RELATED WORK

Operational Transformation is used by most of the synchronous methods [24, 30, 26, 27, 28, 34, 29, 14, 15, 10, 16, 17]. As for asynchronous methods, to our knowledge only

SO6 [20] and MOT2 use it. Its quality is to automatically solve conflicting situations appearing during execution. Indeed, these situations which generally correspond to non-commutative operations, are taken into account before execution when specifying transformation functions. Besides, the use of Operational Transformation generally avoids undoing and then redoing out of order received operations, except in [30, 15].

In the numerous methods that have been proposed to reconcile copies in synchronous environments, especially real time editors, all the copies play the same role and remote operations are generally propagated to copies in a continuous way thanks to causal broadcast. Local operations are always executed without being delayed but the execution of a remote operation must be delayed until it is causally ready (i.e. all operations that causally precede it have been executed at current site). Most methods (i.e. Opt [8], adOpted [24], GOT [30], GOTO [28], SOCT2 [26], SDT [14], LBT [17], ABT [16]) use state vectors or context vectors (i.e. COT [29]) to achieve causal reception and to determine concurrent operations. TIBOT [15] defines rules based on time intervals that constrain operation propagation and synchronization between sites. As for SOCT4 [34] it uses a sequencer.

The way the integration of an operation into a history is achieved depends on the methods employed. AdOpted, SOCT2, GOTO and SDT use both forward and backward transposition and conditions TP1 and TP2 must be met. On the contrary, SOCT4, TIBOT and COT only use forward transposition, and do not have to meet condition TP2 thanks to the fact that the histories associated with the copies are *identical* (i.e. concurrent operations appear in the same order in all histories).

When considering their scalability to P2P systems, synchronous methods suffer from two drawbacks: they need waiting to achieve causal reception and they require mechanisms such as state vectors or a sequencer. State vectors need an entry for each copy in the system. The size of the vectors thus grows in proportion to the number of copies. Besides adding or removing copies becomes complex. As for the sequencer it is generally implemented by a central component.

In asynchronous reconciliation methods, operations are propagated from one site to another when two sites decide to reconcile their copies. A sequence of operations is then transmitted from a site to the other one. According to the methods, only missing operations [23, 22] or more operations, as in MOT2, are transmitted. Respecting causality is achieved thanks to the prefix-property stating that a site that holds an operation also holds all the operations that causally precede it. So a remote operation is naturally causally ready to be integrated as soon as all operations that precede it in the

sequence have been integrated. No waiting is needed to preserve causality.

When a master copy exists, as in MOT1, operations are propagated from one copy to another through the master copy. As a result a global order between operations that also respects the local order on each copy, is directly available and concurrent operations can be determined easily. In the absence of master copy, operations are directly propagated between any two sites and no global order is naturally available. In SO6 [20] a sequencer is used to globally order operations in histories and to determine concurrent operations so as to apply Operational Transformation correctly. In Bayou [23] version vectors are used to characterize copies content and to order write operations. Besides, a master site is required to commit write operations thus providing a total order between committed operations. Note that commitment stabilizes updates and permits limiting the number of operations having to be undone and redone. In MOT2, two operations are considered as concurrent when they immediately follow the prefix common to both histories. The global ordering of operations is obtained by serializing concurrent operations according to their generator sites.

All asynchronous algorithms, except MOT2, require either a central component (master copy, sequencer, master site) or mechanisms such as state vectors, version vectors, etc. The drawbacks of such requirements still have been underlined when addressing scalability to P2P systems.

As for the systems directly proposed for P2P environments, unlike MOT2, they require special feature. In APPA [18], a common storage provided by the P2P system is required to produce a global schedule that will be executed by peers. In WOOT [21], the replicated object is supposed to have a linear structure. The partial ordering relation between the operations is locally maintained at each copy and enclosed to broadcast operations.

VI. CONCLUSION

This paper has studied asynchronous reconciliation of multiple divergent copies of the same object when using Operational Transformation. In this setting, we have proposed an original algorithm, i.e. MOT2, that enables copies to be reconciled pair-wise, at any time, regardless of the pair. MOT2 provides the means to reconcile any number of copies while ensuring their convergence and respecting the potential causal precedence between operations, without discriminating in favour of any particular copy nor requiring either state vectors or a central component. Consequently MOT2 is particularly suitable for a P2P collaborative environment in which communication may be arbitrary or a central component may not be possible. With MOT2 any site can reconcile its copy, *when it wants*, with *any* other site that owns a copy of the object *while achieving convergence of copies*. While

permitting free propagation of update operations, MOT2 guarantees that they will appear in the *same order* in all the histories. It thus ensures, through successive reconciliations, the construction of a *global order* without requiring any ordering mechanism (timestamps, state vectors, sequencer). Finally, the Operational Transformation used (forward transposition only) necessitates verification of condition TP1 only, which is easily met.

VII. APPENDIX

Due to lack of space, only proof of Theorem 5 is given.

Proof of Theorem 5. Two cases must be considered according whether operations op_k and op_l that belong to histories H_i and H_j are concurrent or not.

1) op_k and op_l are not concurrent. They were ordered according to the causality relation. One of the operations, let us say op_l , was generated at site S_{op_l} after execution of the other one at this site. In the history of S_{op_l} , according to definition 2, we can state: $op_k \rightarrow_C op_l$. As from the P1 property, merging preserves the precedence (and therefore the causal ordering precedence), *in all the histories* there will be: $op_k \rightarrow_C op_l$.

2) op_k and op_l are concurrent. They were generated independently of each other in distinct histories. As a result of synchronizing sites, operation op_k which was generated at site S_{op_k} , got propagated to other histories, denoted H^k . In the same way, operation op_l which was generated at site S_{op_l} got propagated to other histories, denoted H^l . Operations op_k and op_l are joined together in the same history when some site from H^l and some site from H^k get synchronized. We will demonstrate that *in any case* op_k and op_l will be ordered by MOT2 in the *same way*. There are again two situations according whether op_k and op_l were generated either from the same state or from different states of the copies.

2.1) op_k and op_l were generated from the same copy state.

As op_k and op_l were generated from the *same state*, then: $\forall op_c: op_c \rightarrow_C op_k$ then $op_c \rightarrow_C op_l$ and conversely: $\forall op_c: op_c \rightarrow_C op_l$ then $op_c \rightarrow_C op_k$. When synchronizing some site S_i from H^k and some site S_j from H^l , MOT2 execution will lead to one of the two following intermediate situations:

either (case 2.1.1): $H_i = H_C.op_k....$ and $H_j = H_C.seq.op_l....$

or (case 2.1.2): $H_i = H_C.seq.op_k....$ and $H_j = H_C.op_l....$

Since $\forall op_c: op_c \rightarrow_C op_k$ then $op_c \rightarrow_C op_l$ and conversely: $\forall op_c: op_c \rightarrow_C op_l$ then $op_c \rightarrow_C op_k$, in both cases we get: $\forall op_c: op_c \rightarrow_C op_k$ or $op_c \rightarrow_C op_l$, $op_c \in H_C$. Then: $\forall op \in seq: op // op_k$ and $op // op_l$. Hereafter, we have to show that, in both cases, synchronization will produce the same result. Let us suppose that $S_{op_k} < S_{op_l}$ (the same reasoning holds for $S_{op_k} > S_{op_l}$).

In case 2.1.1, according to the P2 property, in the resulting history H , we necessarily get: $op_k \rightarrow_H op_l$.

In case 2.1.2, according to the theorem 3, we get: $\forall op \in seq, S_{op} < S_{op_k}$. Based on $S_{op_k} < S_{op_l}$ and taking property P3

into account, in the resulting history H , we necessarily get: $seq \rightarrow_H op_k \rightarrow_H op_l$. Finally, we obtain the following result.

Two concurrent operations op_k and op_l , which were generated from the *same copy state*, are ordered in the resulting history according to their generator site:

if $S_{op_k} < S_{op_l}$ then $op_k \rightarrow_H op_l$

if $S_{op_l} < S_{op_k}$ then $op_l \rightarrow_H op_k$

Therefore, they are ordered in the same way in all the histories.

2.2) op_k and op_l were generated from different copy states. In this case, op_k and op_l are said to be *partially concurrent*. This means there is an operation op_m such that either: $op_m \rightarrow_C op_l$ and op_m and op_k

were generated from the same state,

or: $op_m \rightarrow_C op_k$ and op_m and op_l

were generated from the same state.

Let us consider the first case (similar reasoning is required for the second one). As op_m and op_k are concurrent and were generated from the same state, according to the framed result obtained in 2.1, they will be ordered in the resulting history according to their generator site. Two cases have thus to be investigated depending on the respective values of S_{op_k} and S_{op_m} .

2.2.1) Case where $S_{op_k} < S_{op_m}$. In this case, we get: $op_k \rightarrow_H op_m$. By hypothesis, we also have $op_m \rightarrow_C op_l$. As any history containing op_l also contains all operations which causally precede op_l , we deduce that any history of H^l contains op_m . Consequently, when synchronizing any site of H^l with any site of H^k , in the resulting history we get: $op_k \rightarrow_H op_m \rightarrow_C op_l$ and therefore $op_k \rightarrow_H op_l$.

Finally, by applying a similar reasoning to the second case, we obtain the following result.

Two partially concurrent operations op_k and op_l (i.e. which were generated from different states) are ordered in the resulting history in the following way:

if $\exists op_m: op_m \rightarrow_C op_l$ and

op_m and op_k were generated from the same state

and if $S_{op_k} < S_{op_m}$ then $op_k \rightarrow_H op_l$;

if $\exists op_m: op_m \rightarrow_C op_k$ and

op_m and op_l were generated from the same state

and if $S_{op_l} < S_{op_m}$ then $op_l \rightarrow_H op_k$.

As any history containing an operation also contains all operations which causally precede it, we deduce that any history containing op_l [resp. op_k] also contains op_m . Therefore, op_k and op_l are ordered in the same way in all histories.

2.2.2) Case where $S_{op_k} > S_{op_m}$. In this case, we get in the resulting history: $op_m \rightarrow_H op_k$. More precisely, execution of the MOT2 algorithm will produce the intermediate result:

firstly, $H_i = H_C.op_k....$ and $H_j = H_C.op_m....op_l....$

then, $H_i = H_C.op_m.op_k....$ and $H_j = H_C.op_m....op_l....$

where op_k has been forward transposed with op_m .

The op_k operation obtained after being forward transposed is defined from the state $H_C.op_m$, as if it had been generated from this state. Hereafter, the problem is to determine the respective positions of this new operation op_k and op_l . We are then led again to the beginning of case 2. The reasoning is iterated until obtaining a relation between op_k and op_l relevant to cases 2.1 or 2.2.1.

REFERENCES

- [1] Aberer K., Cudré-Mauroux P., Datta A., Despotovic Z., Hauswirth M., Ponceva M., Schmidt R.; *P-Grid : a self-organizing structured P2P system*; ACM SIGMOD Record, vol.32, n°3, pp.29-33, 2003.
- [2] Berliner B.; *Parallelizing software development*; Proc. USENIX, Washington D.C., 1990.
- [3] Balasubramaniam S., Pierce B.; *What is a file Synchroniser*; Proc. 4th Annual ACM / IEEE International Conference on Mobile Computing and Networking (MobiCom'98), pp.98-108, 1998.
- [4] Chawathe S., Garcia-Molina H.; *Meaningful change detection in structured data*; Proc. ACM International Conference on Management of Data (SIGMOD'97), pp.26-37, 1997.
- [5] Clarke I., Miller S., Hong T., Sandberg O., Wiley B.; *Protecting free expression online with Freenet*; IEEE Internet Computing, vol.6, n°1, pp.40-49, 2002.
- [6] Conradi R., Westfechtel B.; *Version models for software configuration management*; ACM Computing Surveys (CSUR), vol. 30, n°2, pp.232-282, 1998.
- [7] Dourish P.; *The parting of the ways: divergence, data, management and collaborative work*; Proc. 4th European Conference on Computer Supported Cooperative Work (ECSCW'95), pp.215-230, 1995.
- [8] Ellis C.A., Gibbs S.J.; *Concurrency control in groupware systems*; Proc. ACM International Conference on Management of Data (SIGMOD'89), Seattle, pp.399-407, May 1989.
- [9] Estublier J.; *Software configuration management: a roadmap*; Proc. International Conference on Software Engineering (ICSE'00), Limerick, Ireland, pp.279-289, June 2000.
- [10] Ferrié J., Vidot N., Cart M.; *Concurrent undo operations in collaborative environments using operational transformation*; Proc. 12th International Conference on Cooperative Information Systems (CoopIS'04), Cyprus, pp.155-173, October 2004.
- [11] Ignat C., Norrie M.C.; *Operation-based versus state-based merging in asynchronous graphical collaborative editing*; Proc. 6th International Workshop on Collaborative Editing Systems, Chicago, November 2004.
- [12] Ignat C., Norrie M.C.; *Grouping in collaborative graphical editors*; Proc. ACM International Conference on Computer Supported Cooperative Work (CSCW'04), Chicago, November 2004.
- [13] Kermarrec A.M., Rowstron A., Shapiro M., Druschel P.; *The IceCube approach to the reconciliation of divergent replicas*; Proc. 20th ACM Symposium on Principles of Distributed Computing (PODC), Newport R.I., August 2001.
- [14] Li D., Li R.; *Ensuring Content and Intention Consistency in Real-Time Group Editors*; Proc. 24th IEEE International Conference on Distributed Computing Systems (ICDCS'04), Tokyo, Japan, March 2004.
- [15] Li R., Li D., Sun C.; *A time interval based consistency model for interactive groupware applications*; Proc. 10th International Conference on Parallel and Distributed Systems (ICPADS'04), Newport Beach, California, pp. 429-436, July 2004.
- [16] Li R., Li D.; *Commutativity-based concurrency control in groupware*; Proc. 1st International Conference on Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom'05), San Jose, California, December 2005.
- [17] Li R., Li D.; *A landmark-based transformation approach to concurrency control in group editors*; Proc. ACM International Conference on Supporting Group Work (GROUP'05), Sanibel Island, Florida, pp. 284-293, November 2005.
- [18] Martins V., Akbarinia R., Pacitti E., Valduriéz P.; *Reconciliation in the APPA P2P System*; Proc. 12th International Conference on Parallel and Distributed Systems (ICPADS'06), Minneapolis, Minnesota, July 2006.
- [19] Munson J.P., Dewan P.; *A flexible object merging framework*; Proc. ACM International Conference on Computer Supported Cooperative Work (CSCW'94), pp.231-242, 1994.
- [20] Molli P., Oster G., Skaf-Molli H., Imine A.; *Using the transformational approach to build a safe and generic data synchronizer*; Proc. ACM International Conference on Supporting Group Work (GROUP'03), November 2003.
- [21] Oster G., Molli P., Urso P., Imine A.; *Tombstone transformation functions for ensuring consistency in collaborative editing systems*; Proc. 2nd International Conference on Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom'06), Atlanta, Georgia, November 2006.
- [22] Oster G., Urso P., Molli P., Imine A.; *Data consistency for P2P collaborative editing*; Proc. ACM International Conference on Computer Supported Cooperative Work (CSCW'06), Banff, Canada, November 2006.
- [23] Petersen K., Spreitzer M., Terry D., Theimer M., Demers A.; *Flexible update propagation for weakly consistent replication*; Proc. 16th ACM Symposium on Operating Systems Principles (SOSP'97), Saint-Malo, France, pp. 288-301, ACM Press, 1997.
- [24] Ressel M., Nitssche-Ruhland D., Gunzenhäuser R.; *An integrating, transformation-oriented approach to concurrency control and undo in group editors*; Proc. ACM International Conference on Computer Supported Cooperative Work (CSCW'96), Boston, pp. 288-297, November 1996.
- [25] Saito Y., Shapiro M.; *Optimistic replication*; ACM Computing Surveys, vol.37, n°1, pp.42-81, 2005.
- [26] Suleiman M., Cart M., Ferrié J.; *Serialization of concurrent operations in a distributed collaborative environment*; Proc. ACM International Conference on Supporting Group Work (GROUP'97), Phoenix, pp.435-445, November 1997.
- [27] Suleiman M., Cart M., Ferrié J.; *Concurrent operations in a distributed and mobile collaborative environment*; Proc. 14th IEEE International Conference on Data Engineering (IEEE / ICDE'98), Orlando, pp.36-45, February 1998.
- [28] Sun C., Ellis C.S.; *Operational transformation in real-time group editors : issues, algorithms and achievements*; Proc. ACM International Conference on Computer Supported Cooperative Work (CSCW'98), Seattle, pp.59-68, November 1998.
- [29] Sun D., Sun C.; *Operation context and context-based operational transformation*; Proc. ACM International Conference on Computer Supported Cooperative Work (CSCW'06), Banff, Alberta, Canada, pp.279-288, November 2006.
- [30] Sun C., Jia X., Yang Y., Zhang Y.; *A generic operation transformation schema for consistency maintenance in real-time cooperative editing systems*; Proc. ACM International Conference on Supporting Group Work (GROUP'97), Phoenix, pp.425-434, November 1997.
- [31] Tichy W.F.; *RCS - A system for version control*; Software-Practice and Experience, vol.15, n°7, pp.637-654, 1985.
- [32] Toinard C., Florin G., Carrez C.; *A formal method to prove ordering properties of multicast systems*; ACM Operating Systems Review, vol. 33, n°4, pp. 75-89, 1999.
- [33] Torii O., Kimura T., Sego J.; *The consistency control system of XML documents*; Proc. Symposium on Applications and the Internet, January 2003.
- [34] Vidot N., Cart M., Ferrié J., Suleiman M.; *Copies convergence in a distributed real-time collaborative environment*; Proc. ACM International Conference on Computer Supported Cooperative Work (CSCW'00), Philadelphia, Pennsylvania, pp.171-180, December 2000.
- [35] Weihl W. E.; *Commutativity-based concurrency control for abstract data types*; IEEE Transactions on Computers, vol. 37, n° 12, pp.1488-1505, December 1988.